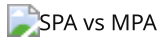


Fundamentos de Routing en SPAs

Objetivos

- Entender la diferencia entre SPA y MPA
 - Comprender cómo funciona el client-side routing
 - Conocer los beneficios y limitaciones del routing en React
-

1. SPA vs MPA



Multi-Page Application (MPA)

En una aplicación tradicional **MPA**:

```
Usuario clickea enlace
    ↓
Navegador solicita nueva página al servidor
    ↓
Servidor procesa y devuelve HTML completo
    ↓
Navegador descarga y renderiza toda la página
    ↓
Se pierde el estado de JavaScript
```

Características MPA:

- Cada navegación = request al servidor
- Página completa se recarga
- El estado se pierde entre navegaciones
- SEO nativo (cada página tiene URL única)
- Tiempo de carga inicial rápido (solo HTML necesario)

Single-Page Application (SPA)

En una **SPA** como las que construimos con React:

```
Usuario clickea enlace
    ↓
JavaScript intercepta el click
    ↓
Actualiza la URL (History API)
    ↓
Renderiza nuevo componente
    ↓
Estado de la aplicación se mantiene
```

Características SPA:

- Una sola carga inicial de HTML/JS/CSS
 - Navegación sin recargar la página
 - Estado se mantiene entre "páginas"
 - Experiencia más fluida (tipo app nativa)
 - Requiere JavaScript habilitado
-

2. ¿Qué es Client-Side Routing?

```
// =====  
// QUÉ: Client-side routing manipula la URL sin request al servidor  
// PARA: Navegar entre vistas sin recargar la página  
// IMPACTO: UX fluida, estado persistente, transiciones suaves  
// =====  
  
// El navegador provee la History API  
window.history.pushState({ page: 1 }, 'Título', '/nueva-url');  
  
// React Router abstrae esto para nosotros  
// En lugar de manipular history directamente, usamos componentes
```

History API del Navegador

```
// =====  
// QUÉ: API nativa del navegador para manipular el historial  
// PARA: Cambiar URL sin recargar la página  
// IMPACTO: Base sobre la que React Router funciona  
// =====  
  
// Agregar entrada al historial  
window.history.pushState(state, title, url);  
  
// Reemplazar entrada actual  
window.history.replaceState(state, title, url);  
  
// Navegar hacia atrás/adelante  
window.history.back();  
window.history.forward();  
window.history.go(-2); // Ir 2 páginas atrás  
  
// Escuchar cambios de navegación  
window.addEventListener('popstate', (event) => {  
  console.log('Navegación:', event.state);  
});
```

3. ¿Por Qué React Router?

Sin una librería de routing, tendríamos que:

```
// =====  
// QUÉ: Routing manual (NO RECOMENDADO)  
// PARA: Entender por qué necesitamos React Router  
// IMPACTO: Código complejo, difícil de mantener  
// =====  
  
import { useState, useEffect } from 'react';  
  
const App: React.FC = () => {  
  const [currentPath, setCurrentPath] = useState(window.location.pathname);
```

```

useEffect(() => {
  // Escuchar navegación del navegador
  const handlePopState = () => {
    setCurrentPath(window.location.pathname);
  };

  window.addEventListener('popstate', handlePopState);
  return () => window.removeEventListener('popstate', handlePopState);
}, []);

// Función para navegar
const navigate = (path: string) => {
  window.history.pushState({}, '', path);
  setCurrentPath(path);
};

// Renderizar según la ruta
return (
  <div>
    <nav>
      <button onClick={() => navigate('/')}>Home</button>
      <button onClick={() => navigate('/about')}>About</button>
      <button onClick={() => navigate('/contact')}>Contact</button>
    </nav>

    {currentPath === '/' && <HomePage />}
    {currentPath === '/about' && <AboutPage />}
    {currentPath === '/contact' && <ContactPage />}
    {![ '/', 'about', '/contact' ].includes(currentPath) && <NotFound />}
  </div>
);
};

```

Problemas de este enfoque:

- ✗ No soporta rutas dinámicas (/users/:id)
- ✗ No maneja query strings (?search=react)
- ✗ No soporta layouts anidados
- ✗ No tiene protección de rutas
- ✗ No maneja loading states
- ✗ Difícil de escalar

4. React Router v6

React Router es la librería estándar para routing en React.

Versión 6: Cambios Importantes

Característica	v5	v6
Definir rutas	<Switch> + <Route>	<Routes> + <Route>
Exact match	exact prop	Por defecto
Rutas anidadas	Componentes separados	<Outlet> + children

Redirección	<Redirect>	<Navigate>
Acceso a params	useParams() + HOC	Solo useParams()
Navegación	useHistory()	useNavigate()

Instalación

```
# Instalar React Router para web
pnpm add react-router-dom

# Tipos incluidos en v6 (no necesitas @types)
```

Arquitectura de React Router v6

```
BrowserRouter (Proveedor de contexto)
├── Routes (Contenedor de rutas)
│   ├── Route path="/" element={<Home />}
│   ├── Route path="/about" element={<About />}
│   └── Route path="/users/:id" element={<UserProfile />}
└── Link / NavLink (Navegación declarativa)
```

5. Tipos de Routers

React Router ofrece diferentes routers según el caso de uso:

```
// =====
// QUÉ: Diferentes tipos de routers disponibles
// PARA: Elegir el apropiado según el deployment
// IMPACTO: URLs limpias vs compatibilidad con hosting
// =====

import {
  BrowserRouter, // URLs limpias: /about
  HashRouter, // URLs con hash: /#/about
  MemoryRouter, // Sin URL (para tests)
  StaticRouter, // Para SSR (Server-Side Rendering)
} from 'react-router-dom';
```

BrowserRouter (Recomendado)

```
// URLs limpias usando History API
// Requiere configuración del servidor para SPA

import { BrowserRouter } from 'react-router-dom';

const App: React.FC = () => (
```

```

    <BrowserRouter>{/* Tu aplicación */}</BrowserRouter>
  );

  // URLs resultantes:
  // https://miapp.com/
  // https://miapp.com/about
  // https://miapp.com/users/123

```

HashRouter (Fallback)

```

// URLs con hash, no requiere configuración del servidor
// Útil para hosting estático simple (GitHub Pages)

import { HashRouter } from 'react-router-dom';

const App: React.FC = () => <HashRouter>{/* Tu aplicación */}</HashRouter>;

// URLs resultantes:
// https://miapp.com/#/
// https://miapp.com/#/about
// https://miapp.com/#/users/123

```

MemoryRouter (Testing)

```

// No manipula la URL del navegador
// Perfecto para tests

import { MemoryRouter } from 'react-router-dom';
import { render } from '@testing-library/react';

test('renders home page', () => {
  render(
    <MemoryRouter initialEntries={['/']}>
      <App />
    </MemoryRouter>,
  );
  // ...assertions
});

```

6. Configuración del Servidor

Para que **BrowserRouter** funcione correctamente, el servidor debe redirigir todas las rutas a `index.html` :

Vite (desarrollo)

```

// vite.config.ts - Ya configurado por defecto
export default defineConfig({
  // El servidor de desarrollo maneja esto automáticamente
});

```

Nginx (producción)

```
# nginx.conf
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Vercel

```
// vercel.json
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }]
}
```

Netlify

```
# netlify.toml
[[redirects]]
  from = "/*"
  to = "/index.html"
  status = 200
```

7. Beneficios y Limitaciones

✅ Beneficios del Client-Side Routing

1. **Experiencia fluida:** Sin recargas de página
2. **Estado persistente:** No se pierde entre navegaciones
3. **Transiciones animadas:** Control total sobre el UI
4. **Carga bajo demanda:** Code splitting por rutas
5. **Modo offline:** Con Service Workers

⚠️ Limitaciones a Considerar

1. **SEO:** Requiere SSR o pre-rendering para crawlers
2. **Carga inicial:** Bundle más grande al inicio
3. **JavaScript requerido:** No funciona sin JS
4. **Configuración servidor:** Necesario para URLs limpias
5. **Historial del navegador:** Complejidad adicional

Recursos Adicionales

- [React Router Docs](#)
- [History API - MDN](#)
- [SPA vs MPA - Patterns](#)

✅ Checklist de Comprensión

- ☐ Entiendo la diferencia entre SPA y MPA
- ☐ Comprendo cómo funciona la History API
- ☐ Sé por qué necesitamos React Router
- ☐ Conozco los diferentes tipos de routers
- ☐ Entiendo la configuración necesaria del servidor

Siguiente: [02-react-router-basico.md](#)

React Router: Configuración Básica

Objetivos

- Configurar React Router en un proyecto TypeScript
 - Entender Routes, Route y sus props
 - Implementar navegación con Link y NavLink
 - Manejar rutas no encontradas (404)
-

1. Setup Inicial

 Estructura de React Router

Instalación

```
# Instalar React Router DOM
pnpm add react-router-dom
```

Configuración en main.tsx

```
// =====
// QUÉ: Envolver la aplicación con BrowserRouter
// PARA: Habilitar routing en toda la aplicación
// IMPACTO: Todos los componentes hijos tienen acceso al router
// =====

// src/main.tsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
);
```

2. Definiendo Rutas

Routes y Route

```
// =====  
// QUÉ: Definir las rutas de la aplicación  
// PARA: Mapear URLs a componentes específicos  
// IMPACTO: Cada URL renderiza el componente correspondiente  
// =====  
  
// src/App.tsx  
import { Routes, Route } from 'react-router-dom';  
  
// Páginas  
import { HomePage } from './pages/HomePage';  
import { AboutPage } from './pages/AboutPage';  
import { ContactPage } from './pages/ContactPage';  
import { NotFoundPage } from './pages/NotFoundPage';  
  
const App: React.FC = () => {  
  return (  
    <div className="app">  
      {/* Navegación (siempre visible) */}  
      <nav>  
        <a href="/">Home</a>  
        <a href="/about">About</a>  
        <a href="/contact">Contact</a>  
      </nav>  
  
      {/* Rutas (cambian según URL) */}  
      <Routes>  
        <Route  
          path="/"  
          element={<HomePage />}  
        />  
        <Route  
          path="/about"  
          element={<AboutPage />}  
        />  
        <Route  
          path="/contact"  
          element={<ContactPage />}  
        />  
        <Route  
          path="*"   
          element={<NotFoundPage />}  
        />  
      </Routes>  
    </div>  
  );  
};  
  
export default App;
```


Props de Route

```
// =====
// QUÉ: Propiedades disponibles en el componente Route
// PARA: Configurar cómo se comporta cada ruta
// IMPACTO: Control granular sobre el matching de rutas
// =====

import { Routes, Route } from 'react-router-dom';

<Routes>
  {/* Ruta básica */}
  <Route
    path="/about"
    element={<AboutPage />}
  />

  {/* Ruta con parámetro dinámico */}
  <Route
    path="/users/:userId"
    element={<UserProfile />}
  />

  {/* Ruta index (hija por defecto) */}
  <Route
    index
    element={<HomePage />}
  />

  {/* Ruta catch-all (404) */}
  <Route
    path="*"
    element={<NotFoundPage />}
  />

  {/* Ruta con case-insensitive */}
  <Route
    path="/About"
    caseSensitive={false}
    element={<AboutPage />}
  />
</Routes>;
```

3. Navegación con Link

⚠ **Importante:** No uses `<a href>` para navegación interna. Causa recarga completa.

Link vs anchor tag

```
// =====
// QUÉ: Componente Link para navegación sin recarga
// PARA: Navegar entre rutas manteniendo el estado de la SPA
// IMPACTO: Experiencia fluida sin perder el estado
// =====
```

```
import { Link } from 'react-router-dom';

const Navigation: React.FC = () => {
  return (
    <nav>
      {/* ❌ MAL: Recarga toda la página */}
      <a href="/about">About (recarga)</a>

      {/* ✅ BIEN: Navegación SPA */}
      <Link to="/about">About</Link>
      <Link to="/">Home</Link>
      <Link to="/contact">Contact</Link>
    </nav>
  );
};
```

Props de Link

```
// =====
// QUÉ: Propiedades del componente Link
// PARA: Personalizar el comportamiento de navegación
// IMPACTO: Control sobre scroll, state y reemplazo de historial
// =====

import { Link } from 'react-router-dom';

const NavigationExamples: React.FC = () => {
  return (
    <nav>
      {/* Navegación básica */}
      <Link to="/products">Productos</Link>

      {/* Con query string */}
      <Link to="/products?category=electronics">Electrónica</Link>

      {/* Con hash */}
      <Link to="/about#team">Nuestro Equipo</Link>

      {/* Objeto to para más control */}
      <Link
        to={{
          pathname: '/products',
          search: '?sort=price',
          hash: '#featured',
        }}>
        Productos Destacados
      </Link>

      {/* Pasar estado (no visible en URL) */}
      <Link
        to="/checkout"
        state={{ fromCart: true, items: 5 }}>
        Checkout
      </Link>
```

```

    {/* Reemplazar historial (no agrega entrada) */}
    <Link
      to="/login"
      replace>
        Login
    </Link>

    {/* Prevenir scroll al top */}
    <Link
      to="/products"
      preventScrollReset>
        Ver más
    </Link>
  </nav>
);
};

```

4. NavLink para Navegación Activa

```

// =====
// QUÉ: NavLink agrega estilos cuando la ruta está activa
// PARA: Indicar visualmente la página actual
// IMPACTO: Mejor UX con feedback visual de ubicación
// =====

import { NavLink } from 'react-router-dom';

const MainNav: React.FC = () => {
  return (
    <nav className="main-nav">
      {/* Clase activa automática */}
      <NavLink to="/">Home</NavLink>
      {/* Agrega clase "active" cuando la URL es "/" */}

      {/* Estilos personalizados según estado */}
      <NavLink
        to="/about"
        className={({ isActive, isPending }) =>
          isActive
            ? 'nav-link active'
            : isPending
              ? 'nav-link pending'
              : 'nav-link'
        }>
        About
      </NavLink>

      {/* Estilos inline según estado */}
      <NavLink
        to="/contact"
        style={({ isActive }) => ({
          fontWeight: isActive ? 'bold' : 'normal',
          color: isActive ? '#61DAFB' : 'ffffff',

```

```

    }>>>
    Contact
  </NavLink>

  {/* Children como función */}
  <NavLink to="/products">
    ({ { isActive }) => (
      <span className={isActive ? 'active' : ''}>
        Productos {isActive && '✓'}
      </span>
    )}
  </NavLink>
</nav>
);
};

```

NavLink con end prop

```

// =====
// QUÉ: Prop end para matching exacto en NavLink
// PARA: Evitar que rutas padre estén "activas" en hijas
// IMPACTO: Solo se marca activa la ruta exacta
// =====

<nav>
  {/* Sin end: "/" estaría activo en /about, /contact, etc. */}
  <NavLink
    to="/"
    end>
    Home
  </NavLink>

  {/* /products estaría activo en /products/123 sin end */}
  <NavLink
    to="/products"
    end>
    Productos
  </NavLink>

  {/* /products activo en /products/123 (sin end) */}
  <NavLink to="/products">Todos los Productos</NavLink>
</nav>

```

5. Estructura de Páginas

Organización Recomendada

```

src/
├─ pages/
│   ├─ HomePage.tsx
│   ├─ AboutPage.tsx
│   ├─ ContactPage.tsx
│   ├─ NotFoundPage.tsx
│   └─ index.ts

```

```
├── components/
│   ├── layout/
│   │   ├── Header.tsx
│   │   ├── Footer.tsx
│   │   └── Navigation.tsx
│   └── ui/
│       └── ...
├── App.tsx
└── main.tsx
```

Ejemplo de Página

```
// =====
// QUÉ: Componente de página tipado
// PARA: Representar una vista completa de la aplicación
// IMPACTO: Componentes reutilizables y mantenibles
// =====

// src/pages/HomePage.tsx
import { Link } from 'react-router-dom';

const HomePage: React.FC = () => {
  return (
    <main className="home-page">
      <h1>Bienvenido a Mi App</h1>
      <p>Esta es la página principal.</p>

      <section className="quick-links">
        <h2>Explora</h2>
        <Link to="/products">Ver Productos</Link>
        <Link to="/about">Sobre Nosotros</Link>
      </section>
    </main>
  );
};

export { HomePage };
```

Barrel Export

```
// src/pages/index.ts
export { HomePage } from './HomePage';
export { AboutPage } from './AboutPage';
export { ContactPage } from './ContactPage';
export { NotFoundPage } from './NotFoundPage';
```

6. Página 404 (Not Found)

```
// =====
// QUÉ: Página para rutas no encontradas
// PARA: Manejar URLs inválidas con gracia
// IMPACTO: Mejor UX, evita pantallas en blanco
```

```
// =====

// src/pages/NotFoundPage.tsx
import { Link, useLocation } from 'react-router-dom';

const NotFoundPage: React.FC = () => {
  const location = useLocation();

  return (
    <main className="not-found-page">
      <h1>404</h1>
      <h2>Página no encontrada</h2>
      <p>
        La ruta <code>{location.pathname}</code> no existe.
      </p>
      <Link to="/">Volver al inicio</Link>
    </main>
  );
};

export { NotFoundPage };
```

Configuración de la ruta 404

```
// src/App.tsx
import { Routes, Route } from 'react-router-dom';
import { HomePage, AboutPage, NotFoundPage } from './pages';

const App: React.FC = () => {
  return (
    <Routes>
      <Route
        path="/"
        element={<HomePage />}
      />
      <Route
        path="/about"
        element={<AboutPage />}
      />
      {/* Ruta catch-all: SIEMPRE al final */}
      <Route
        path="*"
        element={<NotFoundPage />}
      />
    </Routes>
  );
};
```

7. Ejemplo Completo

```
// =====
// QUÉ: Aplicación completa con React Router básico
// PARA: Ver todos los conceptos integrados
```

```
// IMPACTO: Base sólida para aplicaciones más complejas
// =====

// src/App.tsx
import { Routes, Route, NavLink } from 'react-router-dom';
import { HomePage, AboutPage, ContactPage, NotFoundPage } from './pages';
import './App.css';

const App: React.FC = () => {
  return (
    <div className="app">
      {/* Header con navegación */}
      <header className="header">
        <h1>Mi App</h1>
        <nav className="nav">
          <NavLink
            to="/"
            end
            className={({ isActive }) => (isActive ? 'active' : '')}>
            Home
          </NavLink>
          <NavLink
            to="/about"
            className={({ isActive }) => (isActive ? 'active' : '')}>
            About
          </NavLink>
          <NavLink
            to="/contact"
            className={({ isActive }) => (isActive ? 'active' : '')}>
            Contact
          </NavLink>
        </nav>
      </header>

      {/* Contenido principal */}
      <main className="main">
        <Routes>
          <Route
            path="/"
            element={<HomePage />}
          />
          <Route
            path="/about"
            element={<AboutPage />}
          />
          <Route
            path="/contact"
            element={<ContactPage />}
          />
          <Route
            path="*"
            element={<NotFoundPage />}
          />
        </Routes>
      </main>
    </div>
  );
};
```

```

        {/* Footer */}
        <footer className="footer">
            <p>&copy; 2026 Mi App</p>
        </footer>
    </div>
    );
};

export default App;

```

CSS Básico

```

/* src/App.css */
.app {
    min-height: 100vh;
    display: flex;
    flex-direction: column;
}

.header {
    background: #1a1a1a;
    padding: 1rem 2rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.nav {
    display: flex;
    gap: 1rem;
}

.nav a {
    color: #888;
    text-decoration: none;
    padding: 0.5rem 1rem;
    border-radius: 4px;
    transition: all 0.2s;
}

.nav a:hover {
    color: #fff;
}

.nav a.active {
    color: #61dafb;
    background: rgba(97, 218, 251, 0.1);
}

.main {
    flex: 1;
    padding: 2rem;
}

.footer {

```



```
background: #1a1a1a;
padding: 1rem;
text-align: center;
}
```

Recursos Adicionales

- [React Router Tutorial](#)
- [Routes Component API](#)
- [Link Component API](#)

Checklist de Comprensión

- ☐ Sé configurar BrowserRouter en main.tsx
- ☐ Entiendo la diferencia entre Link y anchor tag
- ☐ Puedo usar NavLink con estilos activos
- ☐ Sé configurar una página 404 con path="*"
- ☐ Conozco las props principales de Route

Siguiente: [03-rutas-dinamicas.md](#)

Rutas Dinámicas y Parámetros

Objetivos

- Implementar rutas con parámetros dinámicos
- Usar useParams con TypeScript
- Manejar query strings con useSearchParams
- Implementar navegación programática

1. Rutas con Parámetros

 Rutas Dinámicas

Sintaxis de Parámetros

```
// =====
// QUÉ: Rutas con segmentos dinámicos usando :param
// PARA: Crear páginas que varían según un identificador
// IMPACTO: Una sola definición de ruta para múltiples recursos
// =====

import { Routes, Route } from 'react-router-dom';

const App: React.FC = () => {
  return (
    <Routes>
      {/* Rutas estáticas */}
      <Route
        path="/"
```

```

    element={<HomePage />}
  />
<Route
  path="/products"
  element={<ProductListPage />}
/>

{/* Ruta dinámica con un parámetro */}
<Route
  path="/products/:productId"
  element={<ProductDetailPage />}
/>

{/* Ruta con múltiples parámetros */}
<Route
  path="/users/:userId/posts/:postId"
  element={<UserPostPage />}
/>

{/* Parámetro opcional (con ?) */}
<Route
  path="/search/:query?"
  element={<SearchPage />}
/>
</Routes>
);
};

// URLs que matchean:
// /products/123      → productId = "123"
// /products/abc-def  → productId = "abc-def"
// /users/5/posts/10  → userId = "5", postId = "10"
// /search            → query = undefined
// /search/react      → query = "react"

```

2. useParams Hook

Uso Básico

```

// =====
// QUÉ: Hook para acceder a parámetros de la URL
// PARA: Obtener valores dinámicos de la ruta actual
// IMPACTO: Componentes que reaccionan al parámetro de URL
// =====

import { useParams } from 'react-router-dom';

// Los parámetros siempre son string | undefined
const ProductDetailPage: React.FC = () => {
  const { productId } = useParams();

  // productId es string | undefined
  console.log(productId); // "123" si URL es /products/123

```

```

return (
  <div>
    <h1>Producto: {productId}</h1>
  </div>
);
};

```

Tipado con TypeScript

```

// =====
// QUÉ: Tipar parámetros de URL con TypeScript
// PARA: Obtener autocompletado y validación en tiempo de compilación
// IMPACTO: Código más seguro, menos errores de runtime
// =====

import { useParams } from 'react-router-dom';

// Método 1: Tipo inline
const ProductDetailPage: React.FC = () => {
  const { productId } = useParams<{ productId: string }>();
  // productId es string | undefined

  if (!productId) {
    return <div>Producto no encontrado</div>;
  }

  return <div>Producto: {productId}</div>;
};

// Método 2: Interface separada
interface ProductParams {
  productId: string;
}

const ProductDetailPageTyped: React.FC = () => {
  const { productId } = useParams<ProductParams>();

  return <div>Producto: {productId}</div>;
};

// Método 3: Genérico con Record
const UserPostPage: React.FC = () => {
  const params = useParams<Record<'userId' | 'postId', string>>();

  return (
    <div>
      <p>Usuario: {params.userId}</p>
      <p>Post: {params.postId}</p>
    </div>
  );
};

```

Validación de Parámetros

```
// =====
// QUÉ: Validar y convertir parámetros de URL
// PARA: Asegurar que los parámetros son válidos antes de usar
// IMPACTO: Prevenir errores con IDs inválidos
// =====

import { useParams, Navigate } from 'react-router-dom';
import { useEffect, useState } from 'react';

interface Product {
  id: number;
  name: string;
  price: number;
}

const ProductDetailPage: React.FC = () => {
  const { productId } = useParams<{ productId: string }>();
  const [product, setProduct] = useState<Product | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  // Validar que productId existe y es un número
  const numericId = productId ? parseInt(productId, 10) : NaN;

  if (!productId || isNaN(numericId)) {
    return (
      <Navigate
        to="/products"
        replace
      />
    );
  }

  useEffect(() => {
    const fetchProduct = async () => {
      try {
        setLoading(true);
        const response = await fetch(`/api/products/${numericId}`);

        if (!response.ok) {
          throw new Error('Producto no encontrado');
        }

        const data = await response.json();
        setProduct(data);
      } catch (err) {
        setError(err instanceof Error ? err.message : 'Error desconocido');
      } finally {
        setLoading(false);
      }
    };

    fetchProduct();
  }, [numericId]);

  if (loading) return <p>Cargando...</p>;

```

```

if (error) return <p>Error: {error}</p>;
if (!product) return <p>Producto no encontrado</p>;

return (
  <div>
    <h1>{product.name}</h1>
    <p>Precio: ${product.price}</p>
  </div>
);
};

```

3. useSearchParams Hook

Query Strings

```

// =====
// QUÉ: Hook para manejar query strings (?key=value)
// PARA: Filtros, búsquedas, paginación en la URL
// IMPACTO: Estado compatible vía URL, SEO friendly
// =====

import { useSearchParams } from 'react-router-dom';

// URL: /products?category=electronics&sort=price&page=2
const ProductListPage: React.FC = () => {
  const [searchParams, setSearchParams] = useSearchParams();

  // Leer parámetros
  const category = searchParams.get('category'); // "electronics" | null
  const sort = searchParams.get('sort'); // "price" | null
  const page = searchParams.get('page'); // "2" | null

  // Obtener todos los valores de un parámetro con múltiples valores
  // URL: /products?tag=new&tag=sale
  const tags = searchParams.getAll('tag'); // ["new", "sale"]

  // Verificar si existe un parámetro
  const hasCategory = searchParams.has('category'); // true

  return (
    <div>
      <p>Categoría: {category ?? 'Todas'}</p>
      <p>Ordenar por: {sort ?? 'Relevancia'}</p>
      <p>Página: {page ?? '1'}</p>
    </div>
  );
};

```

Modificar Query Strings

```

// =====
// QUÉ: Actualizar query strings de forma reactiva
// PARA: Filtros interactivos que se reflejan en la URL

```

```

// IMPACTO: URL compatible, estado persistente al recargar
// =====

import { useSearchParams } from 'react-router-dom';

const ProductFilters: React.FC = () => {
  const [searchParams, setSearchParams] = useSearchParams();

  const currentCategory = searchParams.get('category') ?? '';
  const currentSort = searchParams.get('sort') ?? 'relevance';

  // Actualizar un parámetro (reemplaza todos)
  const handleCategoryChange = (category: string) => {
    setSearchParams({ category, sort: currentSort });
  };

  // Actualizar preservando otros parámetros
  const handleSortChange = (sort: string) => {
    setSearchParams((prev) => {
      prev.set('sort', sort);
      return prev;
    });
  };

  // Eliminar un parámetro
  const clearCategory = () => {
    setSearchParams((prev) => {
      prev.delete('category');
      return prev;
    });
  };

  // Limpiar todos los filtros
  const clearAll = () => {
    setSearchParams({});
  };

  return (
    <div className="filters">
      <select
        value={currentCategory}
        onChange={(e) => handleCategoryChange(e.target.value)}>
        <option value="">Todas las categorías</option>
        <option value="electronics">Electrónica</option>
        <option value="clothing">Ropa</option>
        <option value="books">Libros</option>
      </select>

      <select
        value={currentSort}
        onChange={(e) => handleSortChange(e.target.value)}>
        <option value="relevance">Relevancia</option>
        <option value="price">Precio</option>
        <option value="name">Nombre</option>
      </select>
    </div>
  );
};

```

```

        <button onClick={clearAll}>Limpiar filtros</button>
      </div>
    );
  };
};

```

Paginación con searchParams

```

// =====
// QUÉ: Implementar paginación usando query strings
// PARA: Navegación entre páginas de resultados
// IMPACTO: Página actual persistente en URL
// =====

import { useSearchParams } from 'react-router-dom';

interface PaginationProps {
  totalPages: number;
}

const Pagination: React.FC<PaginationProps> = ({ totalPages }) => {
  const [searchParams, setSearchParams] = useSearchParams();

  const currentPage = parseInt(searchParams.get('page') ?? '1', 10);

  const goToPage = (page: number) => {
    setSearchParams((prev) => {
      if (page === 1) {
        prev.delete('page'); // No mostrar page=1 en URL
      } else {
        prev.set('page', String(page));
      }
      return prev;
    });
  };

  return (
    <div className="pagination">
      <button
        onClick={() => goToPage(currentPage - 1)}
        disabled={currentPage <= 1}>
        Anterior
      </button>

      <span>
        Página {currentPage} de {totalPages}
      </span>

      <button
        onClick={() => goToPage(currentPage + 1)}
        disabled={currentPage >= totalPages}>
        Siguiente
      </button>
    </div>
  );
};

```

```
);  
};
```

4. useNavigate Hook

Navegación Programática

```
// =====  
// QUÉ: Hook para navegar programáticamente  
// PARA: Redirigir después de acciones (submit, login, etc.)  
// IMPACTO: Control total sobre cuándo y cómo navegar  
// =====  
  
import { useNavigate } from 'react-router-dom';  
  
const LoginForm: React.FC = () => {  
  const navigate = useNavigate();  
  
  const handleSubmit = async (e: React.FormEvent) => {  
    e.preventDefault();  
  
    try {  
      await loginUser(/* credentials */);  
  
      // Navegar a dashboard después del login  
      navigate('/dashboard');  
    } catch (error) {  
      console.error('Login failed');  
    }  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      {/* campos del formulario */}  
      <button type="submit">Iniciar Sesión</button>  
    </form>  
  );  
};
```

Opciones de useNavigate

```
// =====  
// QUÉ: Opciones avanzadas de navegación  
// PARA: Controlar historial, pasar estado, etc.  
// IMPACTO: Navegación flexible según el caso de uso  
// =====  
  
import { useNavigate } from 'react-router-dom';  
  
const NavigationExamples: React.FC = () => {  
  const navigate = useNavigate();  
  
  // Navegación básica
```



```

const goToHome = () => navigate('/');

// Navegación con replace (no agrega al historial)
const goToLoginReplace = () => navigate('/login', { replace: true });

// Navegación con estado
const goToCheckout = () => {
  navigate('/checkout', {
    state: { fromCart: true, itemCount: 5 },
  });
};

// Navegación relativa
const goBack = () => navigate(-1); // Equivale a history.back()
const goForward = () => navigate(1); // Equivale a history.forward()
const goBackTwo = () => navigate(-2);

// Navegación a ruta relativa al actual
const goToChild = () => navigate('child-route'); // Relativa
const goToSibling = () => navigate('../sibling'); // Subir y bajar

return (
  <div>
    <button onClick={goToHome}>Ir a Home</button>
    <button onClick={goBack}>- Atrás</button>
    <button onClick={goForward}>Adelante →</button>
    <button onClick={goToCheckout}>Ir a Checkout</button>
  </div>
);
};

```

Acceder al Estado de Navegación

```

// =====
// QUÉ: Leer estado pasado durante la navegación
// PARA: Pasar datos entre páginas sin URL
// IMPACTO: Datos sensibles que no deben estar en URL
// =====

import { useLocation, useNavigate } from 'react-router-dom';

// Página origen: pasa estado
const CartPage: React.FC = () => {
  const navigate = useNavigate();

  const proceedToCheckout = () => {
    navigate('/checkout', {
      state: {
        items: cartItems,
        total: calculateTotal(),
        couponApplied: true,
      },
    });
  };
};

```

```

    return <button onClick={proceedToCheckout}>Proceder al pago</button>;
};

// Página destino: lee estado
interface CheckoutState {
  items: CartItem[];
  total: number;
  couponApplied: boolean;
}

const CheckoutPage: React.FC = () => {
  const location = useLocation();
  const navigate = useNavigate();

  // Tipar el estado
  const state = location.state as CheckoutState | null;

  // Validar que tenemos estado (usuario llegó desde el carrito)
  if (!state?.items) {
    // Redirigir si acceden directamente
    return (
      <Navigate
        to="/cart"
        replace
      />
    );
  }

  return (
    <div>
      <h1>Checkout</h1>
      <p>Total: ${state.total}</p>
      <p>Artículos: {state.items.length}</p>
      {state.couponApplied && <p>✓ Cupón aplicado</p>}
    </div>
  );
};

```

5. useLocation Hook

```

// =====
// QUÉ: Hook para obtener información de la URL actual
// PARA: Acceder a pathname, search, hash y state
// IMPACTO: Componentes conscientes de su ubicación
// =====

import { useLocation } from 'react-router-dom';

const LocationDebug: React.FC = () => {
  const location = useLocation();

  // location contiene:
  // {
  //   pathname: "/products/123",

```

```

//  search: "?color=blue",
//  hash: "#reviews",
//  state: { fromHome: true },
//  key: "default"
// }

return (
  <div>
    <p>Pathname: {location.pathname}</p>
    <p>Search: {location.search}</p>
    <p>Hash: {location.hash}</p>
    <p>State: {JSON.stringify(location.state)}</p>
  </div>
);
};

// Caso de uso: Analytics
const usePageTracking = () => {
  const location = useLocation();

  useEffect(() => {
    // Enviar pageview a analytics
    analytics.pageview(location.pathname + location.search);
  }, [location]);
};

```

6. Ejemplo Completo: Lista y Detalle

```

// =====
// QUÉ: Patrón maestro-detalle con rutas dinámicas
// PARA: Listar items y ver detalles de cada uno
// IMPACTO: Navegación natural entre lista y detalle
// =====

// src/App.tsx
import { Routes, Route } from 'react-router-dom';
import { ProductListPage } from './pages/ProductListPage';
import { ProductDetailPage } from './pages/ProductDetailPage';

const App: React.FC = () => (
  <Routes>
    <Route
      path="/products"
      element={<ProductListPage />}
    />
    <Route
      path="/products/:productId"
      element={<ProductDetailPage />}
    />
  </Routes>
);

// src/pages/ProductListPage.tsx
import { Link, useSearchParams } from 'react-router-dom';

```

```

interface Product {
  id: number;
  name: string;
  category: string;
}

const products: Product[] = [
  { id: 1, name: 'Laptop', category: 'electronics' },
  { id: 2, name: 'Camiseta', category: 'clothing' },
  { id: 3, name: 'React Book', category: 'books' },
];

const ProductListPage: React.FC = () => {
  const [searchParams, setSearchParams] = useSearchParams();
  const categoryFilter = searchParams.get('category');

  const filteredProducts = categoryFilter
    ? products.filter((p) => p.category === categoryFilter)
    : products;

  return (
    <div>
      <h1>Productos</h1>

      {/* Filtros */}
      <div>
        <button onClick={() => setSearchParams({})}>Todos</button>
        <button onClick={() => setSearchParams({ category: 'electronics' })}>
          Electrónica
        </button>
        <button onClick={() => setSearchParams({ category: 'clothing' })}>
          Ropa
        </button>
      </div>

      {/* Lista */}
      <ul>
        {filteredProducts.map((product) => (
          <li key={product.id}>
            <Link to={` /products/${product.id}`}>{product.name}</Link>
          </li>
        ))}
      </ul>
    </div>
  );
};

// src/pages/ProductDetailPage.tsx
import { useParams, Link, useNavigate } from 'react-router-dom';

const ProductDetailPage: React.FC = () => {
  const { productId } = useParams<{ productId: string }>();
  const navigate = useNavigate();

  const product = products.find((p) => p.id === Number(productId));

```

```
if (!product) {
  return (
    <div>
      <h1>Producto no encontrado</h1>
      <button onClick={() => navigate('/products')}>
        Volver a productos
      </button>
    </div>
  );
}

return (
  <div>
    <Link to="/products">← Volver</Link>
    <h1>{product.name}</h1>
    <p>Categoría: {product.category}</p>
    <p>ID: {product.id}</p>
  </div>
);
};
```

Recursos Adicionales

- [useParams API](#)
- [useSearchParams API](#)
- [useNavigate API](#)

Checklist de Comprensión

- ☐ Sé definir rutas con parámetros dinámicos (:param)
- ☐ Puedo tipar useParams correctamente
- ☐ Entiendo cómo usar useSearchParams para filtros
- ☐ Sé navegar programáticamente con useNavigate
- ☐ Puedo pasar y leer estado entre navegaciones

Siguiente: [04-navegacion-avanzada.md](#)

Navegación Avanzada: Layouts y Rutas Anidadas

Objetivos

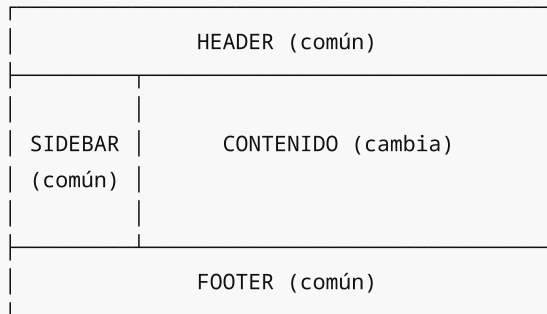
- Implementar layouts reutilizables con Outlet
- Crear rutas anidadas para estructuras complejas
- Usar rutas index para páginas por defecto
- Organizar rutas de forma escalable

1. El Concepto de Layout

 Layouts y Outlet

¿Por qué Layouts?

En aplicaciones reales, muchas páginas comparten estructura:



Sin layouts, repetimos código:

```
// ❌ Sin layouts: repetición de código
const HomePage: React.FC = () => (
  <div>
    <Header />
    <Sidebar />
    <main>Contenido Home</main>
    <Footer />
  </div>
);

const AboutPage: React.FC = () => (
  <div>
    <Header />
    <Sidebar />
    <main>Contenido About</main>
    <Footer />
  </div>
);

// ...repetir en cada página
```

2. Outlet: El "Slot" para Rutas Hijas

```
// =====
// QUÉ: Outlet renderiza el componente de la ruta hija activa
// PARA: Crear layouts que envuelven contenido dinámico
// IMPACTO: DRY - Define la estructura una vez, úsala en múltiples rutas
// =====

import { Outlet } from 'react-router-dom';

// Layout que envuelve las páginas
const MainLayout: React.FC = () => {
  return (
    <div className="layout">
      <Header />
```

```

    <div className="layout-body">
      <Sidebar />

      <main className="content">
        { /* Outlet renderiza la ruta hija activa */ }
        <Outlet />
      </main>
    </div>

    <Footer />
  </div>

  );
};

export { MainLayout };

```

Configuración de Rutas con Layout

```

// =====
// QUÉ: Rutas anidadas bajo un layout común
// PARA: Estructura jerárquica de navegación
// IMPACTO: Código organizado, UI consistente
// =====

import { Routes, Route } from 'react-router-dom';
import { MainLayout } from '../layouts/MainLayout';
import { HomePage, AboutPage, ContactPage } from '../pages';

const App: React.FC = () => {
  return (
    <Routes>
      { /* Layout wrapper */ }
      <Route element={<MainLayout />}>
        { /* Rutas hijas - se renderizan en <Outlet /> */ }
        <Route
          path="/"
          element={<HomePage />}
        />
        <Route
          path="/about"
          element={<AboutPage />}
        />
        <Route
          path="/contact"
          element={<ContactPage />}
        />
      </Route>
    </Routes>
  );
};

// URL "/" → MainLayout + HomePage
// URL "/about" → MainLayout + AboutPage
// URL "/contact" → MainLayout + ContactPage

```

3. Rutas Index

```
// =====  
// QUÉ: Ruta que se muestra por defecto en un layout  
// PARA: Definir qué renderizar cuando no hay ruta hija específica  
// IMPACTO: Evita páginas en blanco en rutas padre  
// =====  
  
import { Routes, Route } from 'react-router-dom';  
  
const App: React.FC = () => {  
  return (  
    <Routes>  
      <Route  
        path="/"  
        element={<MainLayout />}  
        {/* index = se muestra en "/" exactamente */}  
        <Route  
          index  
          element={<HomePage />}  
        />  
        <Route  
          path="about"  
          element={<AboutPage />}  
        />  
      </Route>  
  
      <Route  
        path="/dashboard"  
        element={<DashboardLayout />}  
        {/* index = se muestra en "/dashboard" exactamente */}  
        <Route  
          index  
          element={<DashboardOverview />}  
        />  
        <Route  
          path="analytics"  
          element={<AnalyticsPage />}  
        />  
        <Route  
          path="settings"  
          element={<SettingsPage />}  
        />  
      </Route>  
    </Routes>  
  );  
};  
  
// /dashboard          → DashboardLayout + DashboardOverview  
// /dashboard/analytics → DashboardLayout + AnalyticsPage  
// /dashboard/settings  → DashboardLayout + SettingsPage
```

4. Rutas Anidadas Profundas


```

// =====
// QUÉ: Múltiples niveles de rutas anidadas
// PARA: Estructuras complejas tipo admin/dashboard
// IMPACTO: Cada nivel puede tener su propio layout
// =====

import { Routes, Route, Outlet } from 'react-router-dom';

// Layout de Admin
const AdminLayout: React.FC = () => (
  <div className="admin">
    <AdminSidebar />
    <div className="admin-content">
      <Outlet />
    </div>
  </div>
);

// Layout de Productos (sub-layout)
const ProductsLayout: React.FC = () => (
  <div className="products-section">
    <h2>Gestión de Productos</h2>
    <nav>
      <NavLink to="/admin/products">Lista</NavLink>
      <NavLink to="/admin/products/new">Crear</NavLink>
    </nav>
    <Outlet />
  </div>
);

const App: React.FC = () => {
  return (
    <Routes>
      {/* Rutas públicas */}
      <Route
        path="/"
        element={<PublicLayout />}>
        <Route
          index
          element={<HomePage />}
        />
        <Route
          path="about"
          element={<AboutPage />}
        />
      </Route>

      {/* Rutas de Admin (anidadas) */}
      <Route
        path="/admin"
        element={<AdminLayout />}>
        <Route
          index
          element={<AdminDashboard />}
        />
      </Route>
    </Routes>
  );
};

```

```

    { /* Sub-layout de productos */}
    <Route
      path="products"
      element={<ProductsLayout />}>
      <Route
        index
        element={<ProductList />}
      />
      <Route
        path="new"
        element={<ProductCreate />}
      />
      <Route
        path=":productId"
        element={<ProductDetail />}
      />
      <Route
        path=":productId/edit"
        element={<ProductEdit />}
      />
    </Route>

    { /* Sub-layout de usuarios */}
    <Route
      path="users"
      element={<UsersLayout />}>
      <Route
        index
        element={<UserList />}
      />
      <Route
        path=":userId"
        element={<UserDetail />}
      />
    </Route>
  </Route>
</Routes>
);
};

// Estructura de URLs:
// /admin                → AdminLayout > AdminDashboard
// /admin/products       → AdminLayout > ProductsLayout > ProductList
// /admin/products/new   → AdminLayout > ProductsLayout > ProductCreate
// /admin/products/123   → AdminLayout > ProductsLayout > ProductDetail
// /admin/products/123/edit → AdminLayout > ProductsLayout > ProductEdit
// /admin/users          → AdminLayout > UsersLayout > UserList
// /admin/users/456      → AdminLayout > UsersLayout > UserDetail

```

5. Outlet con Contexto

```

// =====
// QUÉ: Pasar datos del layout a las rutas hijas
// PARA: Compartir estado/funciones entre layout e hijos

```

```

// IMPACTO: Comunicación layout ↔ página sin Context API
// =====

import { Outlet, useOutletContext } from 'react-router-dom';

// Tipos para el contexto
interface DashboardContext {
  user: User;
  sidebarOpen: boolean;
  toggleSidebar: () => void;
}

// Layout que provee contexto
const DashboardLayout: React.FC = () => {
  const [sidebarOpen, setSidebarOpen] = useState(true);
  const { user } = useAuth();

  const toggleSidebar = () => setSidebarOpen((prev) => !prev);

  // Contexto para rutas hijas
  const outletContext: DashboardContext = {
    user,
    sidebarOpen,
    toggleSidebar,
  };

  return (
    <div className={`dashboard ${sidebarOpen ? 'sidebar-open' : ''}`}>
      <Sidebar isOpen={sidebarOpen} />
      <main>
        {/* Pasar contexto a Outlet */}
        <Outlet context={outletContext} />
      </main>
    </div>
  );
};

// Página hija que consume el contexto
const DashboardPage: React.FC = () => {
  const { user, sidebarOpen, toggleSidebar } =
    useOutletContext<DashboardContext>();

  return (
    <div>
      <h1>Bienvenido, {user.name}</h1>
      <button onClick={toggleSidebar}>
        {sidebarOpen ? 'Cerrar' : 'Abrir'} sidebar
      </button>
    </div>
  );
};

// Hook personalizado para tipo seguro
const useDashboardContext = () => {
  return useOutletContext<DashboardContext>();
};

```

6. Múltiples Layouts

```
// =====
// QUÉ: Diferentes layouts según el tipo de página
// PARA: Auth tiene un layout, Dashboard otro, Marketing otro
// IMPACTO: UI adaptada a cada sección de la app
// =====

// Layout público (marketing)
const PublicLayout: React.FC = () => (
  <div className="public-layout">
    <MarketingHeader />
    <Outlet />
    <MarketingFooter />
  </div>
);

// Layout de autenticación (login, register)
const AuthLayout: React.FC = () => (
  <div className="auth-layout">
    <div className="auth-card">
      <Logo />
      <Outlet />
    </div>
  </div>
);

// Layout de aplicación (dashboard)
const AppLayout: React.FC = () => (
  <div className="app-layout">
    <AppHeader />
    <div className="app-body">
      <AppSidebar />
      <main className="app-content">
        <Outlet />
      </main>
    </div>
  </div>
);

// Configuración de rutas
const App: React.FC = () => {
  return (
    <Routes>
      {/* Páginas públicas (marketing) */}
      <Route element={<PublicLayout />}>
        <Route
          path="/"
          element={<LandingPage />}
        />
        <Route
          path="/features"
          element={<FeaturesPage />}
        />
      </Route>
    </Routes>
  );
};
```

```

    <Route
      path="/pricing"
      element={<PricingPage />}
    />
  </Route>

  {/* Páginas de autenticación */}
  <Route element={<AuthLayout />}>
    <Route
      path="/login"
      element={<LoginPage />}
    />
    <Route
      path="/register"
      element={<RegisterPage />}
    />
    <Route
      path="/forgot-password"
      element={<ForgotPasswordPage />}
    />
  </Route>

  {/* Aplicación (requiere auth) */}
  <Route element={<AppLayout />}>
    <Route
      path="/app"
      element={<Dashboard />}
    />
    <Route
      path="/app/projects"
      element={<ProjectsPage />}
    />
    <Route
      path="/app/settings"
      element={<SettingsPage />}
    />
  </Route>

  {/* 404 sin layout */}
  <Route
    path="*"
    element={<NotFoundPage />}
  />
</Routes>
);
};

```

7. Organización de Archivos

Estructura Recomendada

```

src/
├─ layouts/
│   └─ MainLayout.tsx
│   └─ AuthLayout.tsx

```

```
|   ├── DashboardLayout.tsx
|   └── index.ts
└── pages/
    ├── public/
    |   ├── HomePage.tsx
    |   ├── AboutPage.tsx
    |   └── index.ts
    ├── auth/
    |   ├── LoginPage.tsx
    |   ├── RegisterPage.tsx
    |   └── index.ts
    ├── dashboard/
    |   ├── DashboardPage.tsx
    |   ├── SettingsPage.tsx
    |   └── index.ts
    └── index.ts
└── components/
    └── ...
└── routes/
    └── AppRoutes.tsx
└── App.tsx
└── main.tsx
```

Archivo de Rutas Centralizado

```
// =====
// QUÉ: Configuración de rutas en archivo separado
// PARA: Mantener App.tsx limpio y rutas organizadas
// IMPACTO: Fácil mantenimiento y escalabilidad
// =====

// src/routes/AppRoutes.tsx
import { Routes, Route } from 'react-router-dom';
import { MainLayout, AuthLayout, DashboardLayout } from '../layouts';
import { HomePage, AboutPage } from '../pages/public';
import { LoginPage, RegisterPage } from '../pages/auth';
import { DashboardPage, SettingsPage } from '../pages/dashboard';
import { NotFoundPage } from '../pages';

const AppRoutes: React.FC = () => {
  return (
    <Routes>
      {/* Public */}
      <Route element={<MainLayout />}>
        <Route
          index
          element={<HomePage />}
        />
        <Route
          path="about"
          element={<AboutPage />}
        />
      </Route>

      {/* Auth */}
```

```

<Route element={<AuthLayout />}>
  <Route
    path="login"
    element={<LoginPage />}
  />
  <Route
    path="register"
    element={<RegisterPage />}
  />
</Route>

{/* Dashboard */}
<Route
  path="dashboard"
  element={<DashboardLayout />}>
  <Route
    index
    element={<DashboardPage />}
  />
  <Route
    path="settings"
    element={<SettingsPage />}
  />
</Route>

{/* 404 */}
<Route
  path="*"
  element={<NotFoundPage />}
/>
</Routes>
);
};

export { AppRoutes };

// src/App.tsx
import { AppRoutes } from './routes/AppRoutes';

const App: React.FC = () => {
  return <AppRoutes />;
};

export default App;

```

8. Navegación Activa en Layouts

```

// =====
// QUÉ: NavLink con estilos activos en sidebar/header
// PARA: Indicar visualmente la sección actual
// IMPACTO: UX mejorada con feedback visual claro
// =====

import { NavLink } from 'react-router-dom';

```

```
const DashboardSidebar: React.FC = () => {
  // Función helper para clases
  const getLinkClass = ({ isActive }: { isActive: boolean }) =>
    `sidebar-link ${isActive ? 'active' : ''}`;

  return (
    <aside className="sidebar">
      <nav>
        <NavLink
          to="/dashboard"
          end
          className={getLinkClass}>
          📊 Overview
        </NavLink>

        <NavLink
          to="/dashboard/projects"
          className={getLinkClass}>
          📁 Proyectos
        </NavLink>

        <NavLink
          to="/dashboard/team"
          className={getLinkClass}>
          👥 Equipo
        </NavLink>

        <NavLink
          to="/dashboard/settings"
          className={getLinkClass}>
          ⚙️ Configuración
        </NavLink>
      </nav>
    </aside>
  );
};
```

Recursos Adicionales

- [Outlet API](#)
- [Route Nesting](#)
- [Layout Routes](#)

✅ Checklist de Comprensión

- ☐ Entiendo el concepto de Outlet como "slot"
- ☐ Sé crear layouts reutilizables
- ☐ Puedo configurar rutas anidadas
- ☐ Entiendo cuándo usar rutas index
- ☐ Sé pasar contexto a través de Outlet

Siguiente: [05-rutas-protegidas.md](#)

Rutas Protegidas y Autenticación

Objetivos

- Implementar guards para rutas protegidas
- Redirigir usuarios no autenticados
- Manejar roles y permisos
- Crear flujos de autenticación completos

1. Concepto de Rutas Protegidas



¿Por Qué Proteger Rutas?

Algunas páginas solo deben ser accesibles para:

- Usuarios autenticados (dashboard, perfil, settings)
- Usuarios con roles específicos (admin panel)
- Usuarios que cumplen condiciones (onboarding completado)

Usuario no autenticado → /dashboard → Redirigir a /login
Usuario autenticado → /dashboard → Mostrar dashboard
Usuario sin rol admin → /admin → Redirigir a /unauthorized

2. Componente ProtectedRoute

```
// =====  
// QUÉ: Componente que protege rutas verificando autenticación  
// PARA: Redirigir usuarios no autenticados al login  
// IMPACTO: Seguridad en el cliente (complementa seguridad del servidor)  
// =====  
  
import { Navigate, Outlet, useLocation } from 'react-router-dom';  
  
interface ProtectedRouteProps {  
  isAuthenticated: boolean;  
  redirectPath?: string;  
  children?: React.ReactNode;  
}  
  
const ProtectedRoute: React.FC<ProtectedRouteProps> = ({  
  isAuthenticated,  
  redirectPath = '/login',  
  children,  
}) => {  
  const location = useLocation();  
  
  if (!isAuthenticated) {  
    // Guardar la ubicación intentada para redirigir después del login  
    return (  
      <Navigate
```

```

        to={redirectPath}
        state={{ from: location.pathname }}
        replace
      />
    );
  }

  // Si hay children, renderizarlos; si no, usar Outlet para rutas anidadas
  return children ? <>{children}</> : <Outlet />;
};

export { ProtectedRoute };

```

Uso Básico

```

// =====
// QUÉ: Proteger rutas con el componente ProtectedRoute
// PARA: Solo usuarios autenticados acceden a ciertas páginas
// IMPACTO: Flujo de autenticación controlado
// =====

import { Routes, Route } from 'react-router-dom';
import { ProtectedRoute } from '../components/ProtectedRoute';
import { useAuth } from '../hooks/useAuth';

const App: React.FC = () => {
  const { isAuthenticated } = useAuth();

  return (
    <Routes>
      {/* Rutas públicas */}
      <Route
        path="/"
        element={<HomePage />}
      />
      <Route
        path="/login"
        element={<LoginPage />}
      />
      <Route
        path="/register"
        element={<RegisterPage />}
      />

      {/* Rutas protegidas */}
      <Route element={<ProtectedRoute isAuthenticated={isAuthenticated} />>}
        <Route
          path="/dashboard"
          element={<DashboardPage />}
        />
        <Route
          path="/profile"
          element={<ProfilePage />}
        />
      <Route

```

```

        path="/settings"
        element={<SettingsPage />}
      />
    </Route>
  </Routes>
);
};

```

3. Hook useAuth

```

// =====
// QUÉ: Hook para manejar estado de autenticación
// PARA: Centralizar lógica de auth en toda la aplicación
// IMPACTO: Un solo lugar para login, logout, verificación
// =====

import {
  createContext,
  useContext,
  useState,
  useCallback,
  useMemo,
} from 'react';

interface User {
  id: number;
  email: string;
  name: string;
  role: 'user' | 'admin';
}

interface AuthContextType {
  user: User | null;
  isAuthenticated: boolean;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
  checkAuth: () => Promise<void>;
}

const AuthContext = createContext<AuthContextType | null>(null);

const AuthProvider: React.FC<{ children: React.ReactNode }> = ({
  children,
}) => {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  const isAuthenticated = useMemo(() => user !== null, [user]);

  const login = useCallback(async (email: string, password: string) => {
    setIsLoading(true);
    try {
      const response = await fetch('/api/login', {

```

```

        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password }),
    });

    if (!response.ok) {
        throw new Error('Credenciales inválidas');
    }

    const { user, token } = await response.json();
    localStorage.setItem('token', token);
    setUser(user);
} finally {
    setIsLoading(false);
}
}, []);

const logout = useCallback(() => {
    localStorage.removeItem('token');
    setUser(null);
}, []);

const checkAuth = useCallback(async () => {
    const token = localStorage.getItem('token');
    if (!token) {
        setIsLoading(false);
        return;
    }

    try {
        const response = await fetch('/api/me', {
            headers: { Authorization: `Bearer ${token}` },
        });

        if (response.ok) {
            const user = await response.json();
            setUser(user);
        } else {
            localStorage.removeItem('token');
        }
    } catch {
        localStorage.removeItem('token');
    } finally {
        setIsLoading(false);
    }
}, []);

const value = useMemo(
    () => ({ user, isAuthenticated, isLoading, login, logout, checkAuth }),
    [user, isAuthenticated, isLoading, login, logout, checkAuth],
);

return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};

const useAuth = (): AuthContextType => {

```

```

const context = useContext(AuthContext);
if (!context) {
  throw new Error('useAuth debe usarse dentro de AuthProvider');
}
return context;
};

export { AuthProvider, useAuth };

```

4. Redirección Post-Login

```

// =====
// QUÉ: Redirigir al usuario a donde intentaba ir después del login
// PARA: UX fluida, el usuario no pierde su destino
// IMPACTO: Experiencia sin fricciones
// =====

import { useLocation, useNavigate } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth';

interface LocationState {
  from?: string;
}

const LoginPage: React.FC = () => {
  const { login, isLoading } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();

  // Obtener la ruta desde donde fue redirigido
  const state = location.state as LocationState;
  const from = state?.from || '/dashboard';

  const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    const formData = new FormData(e.currentTarget);
    const email = formData.get('email') as string;
    const password = formData.get('password') as string;

    try {
      await login(email, password);
      // Redirigir a donde el usuario quería ir
      navigate(from, { replace: true });
    } catch (error) {
      console.error('Login failed:', error);
    }
  };

  return (
    <div className="login-page">
      <h1>Iniciar Sesión</h1>

      {from !== '/dashboard' && (
        <p className="info">

```

```

        Inicia sesión para acceder a <code>{from}</code>
      </p>
    )}

    <form onSubmit={handleSubmit}>
      <input
        name="email"
        type="email"
        placeholder="Email"
        required
      />
      <input
        name="password"
        type="password"
        placeholder="Contraseña"
        required
      />
      <button
        type="submit"
        disabled={isLoading}>
        {isLoading ? 'Cargando...' : 'Entrar'}
      </button>
    </form>
  </div>
);
};

```

5. Protección por Roles

```

// =====
// QUÉ: Componente que verifica roles específicos
// PARA: Páginas de admin, moderador, etc.
// IMPACTO: Control granular de acceso
// =====

import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth';

type Role = 'user' | 'admin' | 'moderator';

interface RoleProtectedRouteProps {
  allowedRoles: Role[];
  redirectPath?: string;
}

const RoleProtectedRoute: React.FC<RoleProtectedRouteProps> = ({
  allowedRoles,
  redirectPath = '/unauthorized',
}) => {
  const { user, isAuthenticated } = useAuth();
  const location = useLocation();

  // Primero verificar autenticación
  if (!isAuthenticated) {

```

```

    return (
      <Navigate
        to="/login"
        state={{ from: location.pathname }}
        replace
      />
    );
  }

  // Luego verificar rol
  if (!user || !allowedRoles.includes(user.role)) {
    return (
      <Navigate
        to={redirectPath}
        replace
      />
    );
  }

  return <Outlet />;
};

export { RoleProtectedRoute };

```

Uso con Roles

```

// =====
// QUÉ: Configuración de rutas con diferentes niveles de acceso
// PARA: Separar acceso por tipo de usuario
// IMPACTO: Estructura de permisos clara
// =====

import { Routes, Route } from 'react-router-dom';
import { ProtectedRoute } from '../components/ProtectedRoute';
import { RoleProtectedRoute } from '../components/RoleProtectedRoute';

const App: React.FC = () => {
  const { isAuthenticated } = useAuth();

  return (
    <Routes>
      {/* Públicas */}
      <Route
        path="/"
        element={<HomePage />}
      />
      <Route
        path="/login"
        element={<LoginPage />}
      />

      {/* Requiere autenticación (cualquier usuario) */}
      <Route element={<ProtectedRoute isAuthenticated={isAuthenticated} />} />>
        <Route
          path="/dashboard"

```

```

        element={<DashboardPage />}
      />
    <Route
      path="/profile"
      element={<ProfilePage />}
    />
  </Route>

  {/* Solo admin */}
  <Route element={<RoleProtectedRoute allowedRoles={['admin']} />}>
    <Route
      path="/admin"
      element={<AdminLayout />}>
      <Route
        index
        element={<AdminDashboard />}
      />
      <Route
        path="users"
        element={<AdminUsers />}
      />
      <Route
        path="settings"
        element={<AdminSettings />}
      />
    </Route>
  </Route>

  {/* Admin o moderador */}
  <Route
    element={<RoleProtectedRoute allowedRoles={['admin', 'moderator']} />}>
    <Route
      path="/moderation"
      element={<ModerationPage />}
    />
  </Route>

  {/* Página de acceso denegado */}
  <Route
    path="/unauthorized"
    element={<UnauthorizedPage />}
  />

  {/* 404 */}
  <Route
    path="*"
    element={<NotFoundPage />}
  />
</Routes>
);
};

```

6. Loading State durante Verificación


```
// =====
// QUÉ: Mostrar loading mientras se verifica autenticación
// PARA: Evitar flash de contenido incorrecto
// IMPACTO: UX suave sin parpadeos
// =====

import { Navigate, Outlet, useLocation } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth';

const ProtectedRoute: React.FC = () => {
  const { isAuthenticated, isLoading } = useAuth();
  const location = useLocation();

  // Mostrar loading mientras se verifica
  if (isLoading) {
    return (
      <div className="loading-screen">
        <div className="spinner" />
        <p>Verificando autenticación...</p>
      </div>
    );
  }

  if (!isAuthenticated) {
    return (
      <Navigate
        to="/login"
        state={{ from: location.pathname }}
        replace
      />
    );
  }

  return <Outlet />;
};
```

Verificación al Iniciar la App

```
// =====
// QUÉ: Verificar token al cargar la aplicación
// PARA: Mantener sesión si hay token válido
// IMPACTO: Usuario no necesita re-autenticarse
// =====

// src/App.tsx
import { useEffect } from 'react';
import { useAuth } from '../hooks/useAuth';
import { AppRoutes } from '../routes/AppRoutes';

const App: React.FC = () => {
  const { checkAuth, isLoading } = useAuth();

  useEffect(() => {
    checkAuth();
  });
};
```

```

    }, [checkAuth]);

    if (isLoading) {
      return (
        <div className="app-loading">
          <div className="spinner" />
          <p>Cargando aplicación...</p>
        </div>
      );
    }

    return <AppRoutes />;
  };

  export default App;

```

7. Rutas Públicas Solo para No Autenticados

```

// =====
// QUÉ: Redirigir usuarios autenticados fuera de login/register
// PARA: Evitar que usuarios logueados vean páginas de auth
// IMPACTO: Flujo lógico de navegación
// =====

import { Navigate, Outlet } from 'react-router-dom';
import { useAuth } from '../hooks/useAuth';

interface PublicOnlyRouteProps {
  redirectPath?: string;
}

const PublicOnlyRoute: React.FC<PublicOnlyRouteProps> = ({
  redirectPath = '/dashboard',
}) => {
  const { isAuthenticated, isLoading } = useAuth();

  if (isLoading) {
    return <div className="loading">Cargando...</div>;
  }

  // Si ya está autenticado, redirigir al dashboard
  if (isAuthenticated) {
    return (
      <Navigate
        to={redirectPath}
        replace
      />
    );
  }

  return <Outlet />;
};

export { PublicOnlyRoute };

```

```
// Uso
const App: React.FC = () => (
  <Routes>
    { /* Solo para usuarios NO autenticados */ }
    <Route element={<PublicOnlyRoute />}>
      <Route
        path="/login"
        element={<LoginPage />}
      />
      <Route
        path="/register"
        element={<RegisterPage />}
      />
      <Route
        path="/forgot-password"
        element={<ForgotPasswordPage />}
      />
    </Route>

    { /* Solo para usuarios autenticados */ }
    <Route element={<ProtectedRoute />}>
      <Route
        path="/dashboard"
        element={<DashboardPage />}
      />
    </Route>
  </Routes>
);
```

8. Ejemplo Completo

```
// =====
// QUÉ: Aplicación completa con autenticación
// PARA: Ver todos los conceptos integrados
// IMPACTO: Base para aplicaciones con auth real
// =====

// src/App.tsx
import { Routes, Route } from 'react-router-dom';
import { AuthProvider, useAuth } from './hooks/useAuth';
import { ProtectedRoute } from './components/ProtectedRoute';
import { PublicOnlyRoute } from './components/PublicOnlyRoute';
import { RoleProtectedRoute } from './components/RoleProtectedRoute';
import {
  MainLayout,
  AuthLayout,
  DashboardLayout,
  AdminLayout,
} from './layouts';
import {
  HomePage,
  LoginPage,
  RegisterPage,
```

```
DashboardPage,
ProfilePage,
AdminPage,
UnauthorizedPage,
NotFoundPage,
} from './pages';
import { useEffect } from 'react';

const AppContent: React.FC = () => {
  const { checkAuth, isLoading, isAuthenticated } = useAuth();

  useEffect(() => {
    checkAuth();
  }, [checkAuth]);

  if (isLoading) {
    return <div className="app-loading">Cargando...</div>;
  }

  return (
    <Routes>
      { /* Páginas públicas */ }
      <Route element={<MainLayout />}>
        <Route
          path="/"
          element={<HomePage />}
        />
      </Route>

      { /* Auth (solo no autenticados) */ }
      <Route element={<PublicOnlyRoute />}>
        <Route element={<AuthLayout />}>
          <Route
            path="/login"
            element={<LoginPage />}
          />
          <Route
            path="/register"
            element={<RegisterPage />}
          />
        </Route>
      </Route>

      { /* Dashboard (requiere auth) */ }
      <Route element={<ProtectedRoute isAuthenticated={isAuthenticated} />}>
        <Route element={<DashboardLayout />}>
          <Route
            path="/dashboard"
            element={<DashboardPage />}
          />
          <Route
            path="/profile"
            element={<ProfilePage />}
          />
        </Route>
      </Route>
    </Route>
  );
}
```

```
    { /* Admin (requiere rol admin) */ }
    <Route element={<RoleProtectedRoute allowedRoles={['admin']} />}>
      <Route
        path="/admin"
        element={<AdminLayout />}>
        <Route
          index
          element={<AdminPage />}
        />
      </Route>
    </Route>

    { /* Páginas de error */ }
    <Route
      path="/unauthorized"
      element={<UnauthorizedPage />}
    />
    <Route
      path="*"
      element={<NotFoundPage />}
    />
  </Routes>
);
};

const App: React.FC = () => (
  <AuthProvider>
    <AppContent />
  </AuthProvider>
);

export default App;
```

Recursos Adicionales

- [React Router Auth Example](#)
- [Navigate Component](#)
- [Protected Routes Pattern](#)

Checklist de Comprensión

- ☐ Entiendo el patrón de ProtectedRoute
- ☐ Sé redirigir usuarios no autenticados
- ☐ Puedo implementar protección por roles
- ☐ Entiendo el flujo de redirección post-login
- ☐ Sé manejar loading states durante verificación

Final de la teoría · Semana 07

Ejercicio 01: Configuración Básica de React Router

Objetivo

Configurar React Router v6 en una aplicación React con TypeScript, creando las primeras rutas básicas.

Instrucciones

En este ejercicio configurarás React Router desde cero. El código está comentado para que lo descomentes paso a paso.

Paso 1: Instalar Dependencias

```
pnpm add react-router-dom
```

Paso 2: Configurar BrowserRouter

Abre `starter/main.tsx` y descomenta el código para envolver la app con BrowserRouter.

Paso 3: Crear Páginas Básicas

Abre los archivos en `starter/pages/` y descomenta el código de cada página.

Paso 4: Configurar Routes

Abre `starter/App.tsx` y descomenta la configuración de rutas.

Paso 5: Probar la Navegación

Ejecuta la aplicación y navega manualmente cambiando la URL:

- `http://localhost:5173/`
- `http://localhost:5173/about`
- `http://localhost:5173/contact`

Archivos

```
ejercicio-01/
├── README.md
├── starter/
│   ├── main.tsx
│   ├── App.tsx
│   └── pages/
│       ├── HomePage.tsx
│       ├── AboutPage.tsx
│       └── ContactPage.tsx
└── solution/
    ├── main.tsx
    ├── App.tsx
    └── pages/
        ├── HomePage.tsx
        ├── AboutPage.tsx
        └── ContactPage.tsx
```

Criterios de Éxito

- ☐ BrowserRouter envuelve la aplicación

- ☐ Tres rutas configuradas (/ , /about, /contact)
- ☐ Cada ruta renderiza su página correspondiente
- ☐ No hay errores en consola

Siguiente: [Ejercicio 02 - Navegación con Link](#)



Ejercicio 02: Navegación con Link y NavLink



Objetivo

Implementar navegación declarativa usando los componentes Link y NavLink de React Router.



Instrucciones

Paso 1: Crear un Navigation Component

Abre `starter/components/Navigation.tsx` y descomenta el código para crear un menú con Link.

Paso 2: Agregar estilos activos con NavLink

Modifica el componente para usar NavLink en lugar de Link, aplicando estilos cuando la ruta está activa.

Paso 3: Integrar el Navigation en App

Abre `starter/App.tsx` y agrega el componente Navigation al layout.



Archivos

```
ejercicio-02/
├── README.md
├── starter/
│   ├── App.tsx
│   ├── components/
│   │   └── Navigation.tsx
│   ├── styles/
│   │   └── Navigation.css
└── solution/
    ├── App.tsx
    ├── components/
    │   └── Navigation.tsx
    ├── styles/
    │   └── Navigation.css
```



Criterios de Éxito

- ☐ Navigation muestra links a todas las rutas
- ☐ Los links navegan sin recargar la página
- ☐ NavLink muestra estilo activo en la ruta actual
- ☐ La navegación es accesible (keyboard navigation)

Siguiente: [Ejercicio 03 - Rutas Dinámicas](#)



Ejercicio 03: Rutas Dinámicas con useParams



Objetivo

Implementar rutas con parámetros dinámicos usando useParams y useSearchParams.



Instrucciones

Paso 1: Crear Ruta con Parámetro

Abre `starter/App.tsx` y agrega una ruta dinámica `/products/:productId`.

Paso 2: Usar useParams

Abre `starter/pages/ProductPage.tsx` y usa useParams para obtener el productId.

Paso 3: Implementar useSearchParams

Abre `starter/pages/ProductListPage.tsx` e implementa filtros con query strings.

Paso 4: Navegar con parámetros

Crea links que naveguen a productos específicos y con filtros.



Archivos

```
ejercicio-03/
├── README.md
├── starter/
│   ├── App.tsx
│   └── pages/
│       ├── ProductListPage.tsx
│       └── ProductPage.tsx
└── solution/
    ├── App.tsx
    └── pages/
        ├── ProductListPage.tsx
        └── ProductPage.tsx
```



Criterios de Éxito

- ☐ Ruta dinámica captura el parámetro productId
 - ☐ useParams obtiene el valor del parámetro
 - ☐ useSearchParams maneja filtros en la URL
 - ☐ Navegación funciona con parámetros
-

Siguiente: [Ejercicio 04 - Layouts](#)



Ejercicio 04: Layouts Anidados con Outlet



Objetivo

Implementar layouts reutilizables con el componente Outlet para rutas anidadas.

Instrucciones

Paso 1: Crear un Layout Principal

Abre `starter/layouts/MainLayout.tsx` y crea un layout con Header, contenido y Footer.

Paso 2: Usar Outlet para Contenido Dinámico

El Outlet renderiza la ruta hija activa dentro del layout.

Paso 3: Configurar Rutas Anidadas

Abre `starter/App.tsx` y configura las rutas para usar el layout.

Paso 4: Crear un Sub-Layout (Dashboard)

Implementa un layout secundario para la sección dashboard.

Archivos

```
ejercicio-04/
├── README.md
├── starter/
│   ├── App.tsx
│   └── layouts/
│       ├── MainLayout.tsx
│       └── DashboardLayout.tsx
└── solution/
    ├── App.tsx
    └── layouts/
        ├── MainLayout.tsx
        └── DashboardLayout.tsx
```

Criterios de Éxito

- ☐ MainLayout muestra Header y Footer en todas las páginas
 - ☐ Outlet renderiza el contenido de cada ruta
 - ☐ Rutas anidadas funcionan correctamente
 - ☐ DashboardLayout tiene su propia estructura
-

Siguiente: [Ejercicio 05 - Rutas Protegidas](#)

Ejercicio 05: Rutas Protegidas

Objetivo

Implementar protección de rutas para páginas que requieren autenticación.

Instrucciones

Paso 1: Crear el componente ProtectedRoute

Abre `starter/components/ProtectedRoute.tsx` e implementa la lógica de protección.

Paso 2: Implementar redirección al login

Si el usuario no está autenticado, redirigir a `/login` guardando la URL destino.

Paso 3: Configurar rutas protegidas

Abre `starter/App.tsx` y envuelve las rutas que requieren autenticación.

Paso 4: Implementar redirección post-login

Después del login, redirigir a la página que el usuario quería visitar.

Archivos

```
ejercicio-05/
├── README.md
├── starter/
│   ├── App.tsx
│   ├── components/
│   │   └── ProtectedRoute.tsx
│   ├── pages/
│   │   ├── LoginPage.tsx
│   │   └── DashboardPage.tsx
│   └── hooks/
│       └── useAuth.tsx
└── solution/
    ├── App.tsx
    ├── components/
    │   └── ProtectedRoute.tsx
    ├── pages/
    │   ├── LoginPage.tsx
    │   └── DashboardPage.tsx
    └── hooks/
        └── useAuth.tsx
```

Criterios de Éxito

- ☐ ProtectedRoute redirige usuarios no autenticados
- ☐ La URL destino se guarda en el state de navegación
- ☐ Después del login, se redirige a la URL guardada
- ☐ Las rutas públicas funcionan sin autenticación

Fin de ejercicios · Semana 07



Proyecto Semanal: Sistema de Navegación Multi-página



Objetivo

Implementar un sistema de navegación completo aplicando todos los conceptos de React Router v6 aprendidos esta semana, adaptado a tu dominio asignado.

Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio]

Requisitos Funcionales

1. Estructura de Navegación

Implementa las siguientes rutas según tu dominio:

Ruta	Descripción	Ejemplo (Biblioteca)
/	Página principal	Landing page
/items	Lista de elementos	Lista de libros
/items/:id	Detalle de elemento	Detalle de libro
/dashboard	Panel de gestión (protegido)	Dashboard de admin
/login	Página de login	Login
/about	Información	Acerca de
*	404 Not Found	Página no encontrada

2. Características Requeridas

- ☐ **Navegación con NavLink** - Menú con indicador de ruta activa
- ☐ **Rutas dinámicas** - Página de detalle con `useParams`
- ☐ **Query strings** - Filtros en la lista con `useSearchParams`
- ☐ **Layout compartido** - Header y Footer comunes con `Outlet`
- ☐ **Rutas protegidas** - Dashboard solo para usuarios autenticados
- ☐ **Navegación programática** - Redirección después de acciones
- ☐ **Página 404** - Manejo de rutas no encontradas

3. Estructura de Componentes

```
src/
├── layouts/
│   └── MainLayout.tsx      # Header + Outlet + Footer
├── components/
│   ├── Navigation.tsx     # NavLink
│   └── ProtectedRoute.tsx # Guard de autenticación
├── pages/
│   ├── HomePage.tsx
│   ├── ItemListPage.tsx   # Lista con filtros
│   ├── ItemDetailPage.tsx # Detalle con useParams
│   ├── DashboardPage.tsx  # Protegida
│   ├── LoginPage.tsx
│   ├── AboutPage.tsx
│   └── NotFoundPage.tsx
└── hooks/
```

	└─ useAuth.tsx	# Estado de autenticación
└─	App.tsx	# Configuración de rutas

Ejemplos de Adaptación por Dominio

Biblioteca

- Items: Libros
- Filtros: género, autor, disponibilidad
- Dashboard: Gestión de préstamos

Farmacia

- Items: Medicamentos
- Filtros: categoría, precio, receta requerida
- Dashboard: Inventario y ventas

Gimnasio

- Items: Clases o equipos
- Filtros: horario, nivel, instructor
- Dashboard: Membresías activas

Restaurante

- Items: Platillos del menú
- Filtros: categoría, precio, vegano
- Dashboard: Pedidos activos

Archivos Starter

```

proyecto/
├─ README.md
├─ starter/
│  └─ src/
│     └─ App.tsx
│     └─ main.tsx
│     └─ layouts/
│        └─ MainLayout.tsx
│     └─ components/
│        └─ Navigation.tsx
│        └─ ProtectedRoute.tsx
│     └─ pages/
│        └─ ... (archivos con TODOs)
│     └─ hooks/
│        └─ useAuth.tsx
├─ package.json
├─ vite.config.ts
└─ solution/
   └─ ... (solución completa)

```

Criterios de Evaluación

Conocimiento (30%)

- ☐ Comprende la diferencia entre Link y NavLink

- ☐ Entiende useParams vs useSearchParams
- ☐ Sabe explicar el propósito de Outlet

Desempeño (40%)

- ☐ Configura BrowserRouter correctamente
- ☐ Implementa rutas anidadas con layouts
- ☐ Usa hooks de navegación apropiadamente

Producto (30%)

- ☐ Navegación funcional sin recargas
- ☐ Filtros reflejados en URL
- ☐ Rutas protegidas funcionando
- ☐ Código limpio y tipado



Entregables

1. **Código funcional** - Aplicación corriendo sin errores
2. **README personalizado** - Documentación de tu dominio
3. **Capturas de pantalla** - Navegación funcionando



Referencias

- [React Router v6 Docs](#)
- [Teoría Semana 07](#)
- [Ejercicios Semana 07](#)

Proyecto Final · Semana 07 · React Router v6



Glosario - Semana 07

React Router v6 con TypeScript

A

Active Route (Ruta Activa)

La ruta que actualmente coincide con la URL del navegador. En React Router, `NavLink` puede detectar si es la ruta activa para aplicar estilos diferentes.

```
// NavLink aplica 'active' automáticamente a la ruta activa
<NavLink to="/about" className={({ isActive }) => isActive ? 'active' : ''}>
  About
</NavLink>
```

B

BrowserRouter

Componente que utiliza la History API del navegador para mantener la UI sincronizada con la URL. Es el router más común para aplicaciones web.

```
import { BrowserRouter } from 'react-router-dom';

<BrowserRouter>
  <App />
</BrowserRouter>
```

Back Navigation

Navegación hacia atrás en el historial del navegador, que se puede implementar programáticamente.

```
const navigate = useNavigate();
navigate(-1); // Ir hacia atrás
navigate(-2); // Ir 2 páginas atrás
```

C

Client-Side Routing (Enrutamiento del Lado del Cliente)

Técnica donde la navegación entre páginas es manejada por JavaScript en el navegador, sin hacer peticiones al servidor para cada cambio de página.

Catch-All Route (Ruta Comodín)

Ruta que captura todas las URLs que no coinciden con ninguna otra ruta definida. Se usa para páginas 404.

```
<Route path="*" element={<NotFoundPage />} />
```

D

Dynamic Route (Ruta Dinámica)

Ruta que contiene segmentos variables, representados con `:` seguido de un nombre de parámetro.

```
// :bookId es un segmento dinámico
<Route path="/books/:bookId" element={<BookDetail />} />
```

Deep Linking

Capacidad de enlazar directamente a una página específica dentro de una aplicación, incluyendo parámetros de URL y query strings.

E

element prop

Propiedad de `<Route>` que especifica el componente React a renderizar cuando la ruta coincide.

```
<Route path="/about" element={<AboutPage />} />
```

end prop

Propiedad de `NavLink` que indica que solo debe marcarse como activo si la ruta coincide exactamente.

```
<NavLink to="/" end>Home</NavLink>
```

G

Guard (Guardia de Ruta)

Componente o lógica que protege rutas verificando condiciones antes de permitir el acceso.

```
const ProtectedRoute = ({ isAuthenticated, children }) => {  
  if (!isAuthenticated) return <Navigate to="/login" />;  
  return children;  
};
```

H

HashRouter

Router que usa el hash (#) de la URL para simular diferentes rutas. Útil cuando no se puede configurar el servidor para manejar rutas.

```
import { HashRouter } from 'react-router-dom';  
// URLs lucen como: example.com/#!/about
```

History API

API del navegador que permite manipular el historial de navegación sin recargar la página. Es la base de `BrowserRouter`.

I

Index Route (Ruta Índice)

Ruta hija que se renderiza cuando la URL coincide exactamente con la ruta padre.

```
<Route path="dashboard" element={<DashboardLayout />}>  
  <Route index element={<DashboardHome />} /> { /* Se muestra en /dashboard */}  
  <Route path="settings" element={<Settings />} />  
</Route>
```

isActive

Propiedad booleana disponible en el callback de `className` o `style` de `NavLink` que indica si la ruta está activa.

L

Layout Route (Ruta de Layout)

Ruta que define un layout compartido para sus rutas hijas, usando `<Outlet>` para renderizar el contenido dinámico.

<Link>

Componente para navegación declarativa que previene la recarga de página.

```
<Link to="/about">About</Link>
<Link to="/books/123">Book Detail</Link>
```

Loader

Función en React Router v6.4+ que carga datos antes de renderizar una ruta.

M

MemoryRouter

Router que mantiene el historial en memoria, sin modificar la URL del navegador. Útil para testing.

MPA (Multi-Page Application)

Aplicación tradicional donde cada navegación resulta en una petición al servidor y recarga completa de la página.

N

<Navigate>

Componente que redirige automáticamente a otra ubicación cuando se renderiza.

```
if (!isAuthenticated) {
  return <Navigate to="/login" replace />;
}
```

<NavLink>

Versión especial de `<Link>` que sabe si está "activa" basándose en la URL actual.

Nested Routes (Rutas Anidadas)

Rutas definidas dentro de otras rutas, creando jerarquías de componentes.

```
<Route path="dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
  <Route path="settings" element={<Settings />} />
</Route>
```

O

<Outlet>

Componente que renderiza el contenido de la ruta hija actual dentro de un layout padre.

```
const Layout = () => (
  <div>
    <Header />
    <Outlet /> { /* Aquí se renderiza la ruta hija */}
    <Footer />
  </div>
);
```

P

Path Parameters (Parámetros de Ruta)

Segmentos dinámicos de una URL que se extraen usando `useParams`.

```
// URL: /books/123
const { bookId } = useParams<{ bookId: string }>();
// bookId = "123"
```

Protected Route (Ruta Protegida)

Ruta que requiere autenticación u otra condición para ser accesible.

Programmatic Navigation

Navegación ejecutada mediante código JavaScript, no por clicks del usuario.

```
const navigate = useNavigate();
navigate('/dashboard');
```

Q

Query String (Cadena de Consulta)

Parte de la URL después del `?` que contiene pares clave-valor para pasar datos.

```
// URL: /books?genre=fiction&sort=title
const [searchParams] = useSearchParams();
const genre = searchParams.get('genre'); // "fiction"
```

R

`replace`

Opción de navegación que reemplaza la entrada actual del historial en lugar de agregar una nueva.

```
navigate('/home', { replace: true });
<Navigate to="/login" replace />
```

`<Route>`

Componente que define una relación entre una URL y el componente a renderizar.

`<Routes>`

Contenedor que agrupa múltiples `<Route>` y selecciona el que mejor coincide con la URL actual.

S

SPA (Single-Page Application)

Aplicación que carga una sola página HTML y actualiza dinámicamente el contenido sin recargar.

state

Datos adicionales que se pueden pasar durante la navegación, accesibles en la página de destino.

```
// Enviar state
navigate('/dashboard', { state: { from: '/login' } });

// Recibir state
const location = useLocation();
const { from } = location.state || {};
```

U

useLocation

Hook que retorna el objeto location actual, incluyendo pathname, search, hash y state.

```
const location = useLocation();
console.log(location.pathname); // "/books"
console.log(location.search); // "?genre=fiction"
```

useNavigate

Hook que retorna una función para navegación programática.

```
const navigate = useNavigate();
navigate('/about');
navigate(-1); // Atrás
```

useParams

Hook que retorna un objeto con los parámetros de ruta de la URL actual.

```
// Ruta: /books/:bookId
const { bookId } = useParams<{ bookId: string }>();
```

useSearchParams

Hook que permite leer y modificar los query parameters de la URL.

```
const [searchParams, setSearchParams] = useSearchParams();
const genre = searchParams.get('genre');
setSearchParams({ genre: 'fiction' });
```

URL Parameters

Ver "Path Parameters" y "Query String".

W

Wildcard Route

Ver "Catch-All Route".

Símbolos

: (Dos Puntos)

Prefijo que indica un segmento dinámico en una ruta.

```
<Route path="/users/:userId" element={<UserProfile />} />
```

* (Asterisco)

Coincide con cualquier ruta. Se usa para páginas 404.

```
<Route path="*" element={<NotFound />} />
```

? (Interrogación)

En URLs, inicia la cadena de query parameters.

```
/search?query=react&page=1
```

Glosario · Semana 07 · React Router v6

Semana 07: React Router v6

Objetivo de la Semana

Dominar la navegación en aplicaciones React SPA utilizando React Router v6 con TypeScript, implementando rutas dinámicas, protegidas y layouts anidados.

Contenidos

Teoría

Archivo	Tema	Duración
01-fundamentos-routing.md	SPA vs MPA, conceptos de routing	30 min
02-react-router-basico.md	Configuración, Routes, Link	45 min
03-rutas-dinamicas.md	Parámetros, useParams, useSearchParams	45 min
04-navegacion-avanzada.md	Layouts, Outlet, rutas anidadas	45 min
05-rutas-protegidas.md	Guards, redirección, auth	45 min

Ejercicios

Ejercicio	Enfoque	Duración
ejercicio-01	Setup inicial de React Router	45 min
ejercicio-02	Link, NavLink, navegación	45 min
ejercicio-03	useParams, rutas dinámicas	45 min

ejercicio-04	Layouts con Outlet	45 min
ejercicio-05	Autenticación y guards	45 min

Proyecto

Proyecto	Descripción
Proyecto Semanal	Sistema de navegación multi-página con auth



Distribución del Tiempo

Actividad	Horas
Teoría	3.0
Ejercicios guiados	3.75
Proyecto semanal	1.25
Total	8



Resultados de Aprendizaje

Al finalizar esta semana, serás capaz de:

- ✓ Configurar React Router v6 en proyectos TypeScript
- ✓ Implementar navegación con Link y NavLink
- ✓ Crear rutas dinámicas con parámetros tipados
- ✓ Construir layouts reutilizables con Outlet
- ✓ Manejar query strings con useSearchParams
- ✓ Implementar rutas protegidas con autenticación
- ✓ Usar useNavigate para navegación programática
- ✓ Gestionar estados de carga con loader/action

Entregable

Entregable	Peso
Proyecto Semanal 📦	100%



Nota: La teoría y los ejercicios son recursos de aprendizaje para prepararte para el proyecto. No son entregables evaluados.

✓ Checklist de Entrega

Preparación (No evaluada)

- ☐ Leer material teórico de la semana
- ☐ Completar ejercicios guiados para practicar

Entregable (Evaluado - 100%)

- ☐ Proyecto semanal completado
- ☐ BrowserRouter configurado correctamente
- ☐ Mínimo 5 rutas implementadas
- ☐ Al menos 2 rutas con parámetros dinámicos

- ☐ Layout principal con navegación
- ☐ Ruta protegida con redirección
- ☐ Página 404 personalizada
- ☐ Código TypeScript sin errores
- ☐ README con instrucciones del dominio

Recursos de la Semana

- [Recursos adicionales](#)
- [Glosario de términos](#)

Navegación

Anterior	Siguiente
← Semana 06	Semana 08 →

Etapa

Etapa 3: React Intermedio · Semana 1 de 5

Bootcamp React · Semana 07

Rúbrica de Evaluación - Semana 07

React Router v6

Distribución de Evaluación

Componente	Peso
Proyecto Semanal	100%

Recursos de Aprendizaje (No Evaluados)

La teoría y los ejercicios son recursos de preparación para el proyecto. No son entregables evaluados, pero se recomienda completarlos antes de iniciar el proyecto.

Teoría

- Fundamentos de routing en SPAs
- Configuración de React Router v6
- Rutas dinámicas y parámetros
- Layouts y rutas anidadas
- Rutas protegidas y autenticación

Ejercicios Guiados

- Ejercicio 01: Configuración básica de React Router
- Ejercicio 02: Navegación con Link y NavLink
- Ejercicio 03: Rutas dinámicas con useParams

- Ejercicio 04: Layouts anidados con Outlet
- Ejercicio 05: Rutas protegidas con guards

Proyecto (100%)

Funcionalidad (40 puntos)

Nivel	Descripción	Puntos
Excelente	Navegación completa, todas las rutas funcionan, sin errores	36-40
Bueno	Navegación funciona con mínimos bugs	28-35
Suficiente	Navegación básica funciona, algunos problemas menores	20-27
Insuficiente	Navegación con errores críticos o funcionalidades incompletas	0-19

Criterios específicos:

- BrowserRouter configurado correctamente (8 pts)
- Mínimo 5 rutas implementadas (8 pts)
- Al menos 2 rutas con parámetros dinámicos usando useParams (8 pts)
- Ruta protegida con redirección funcional (8 pts)
- Página 404 personalizada (8 pts)

Adaptación al Dominio (35 puntos)

Nivel	Descripción	Puntos
Excelente	Rutas coherentes con el dominio, entidades bien representadas	32-35
Bueno	Dominio aplicado correctamente con adaptaciones menores	25-31
Suficiente	Dominio reconocible pero con inconsistencias	18-24
Insuficiente	Dominio no aplicado o rutas genéricas sin contexto	0-17

Criterios específicos:

- Rutas nombradas según entidades del dominio (10 pts)
- Navegación lógica para el contexto del negocio (10 pts)
- Parámetros dinámicos relevantes al dominio (8 pts)
- README documenta el dominio y sus rutas (7 pts)

Calidad del Código (25 puntos)

Nivel	Descripción	Puntos
Excelente	TypeScript estricto, código limpio, rutas bien organizadas	23-25
Bueno	Tipado correcto, código legible y mantenible	18-22
Suficiente	Funciona pero con tipos any o código desorganizado	13-17
Insuficiente	Sin tipado, código difícil de mantener o con errores	0-12

Criterios específicos:

- TypeScript sin errores de compilación (8 pts)
- Parámetros de ruta tipados correctamente (6 pts)
- Componentes organizados y reutilizables (6 pts)

- Layout principal con Outlet implementado (5 pts)



Escala de Calificación

Rango	Calificación	Descripción
90-100	A	Excelente
80-89	B	Bueno
70-79	C	Suficiente
60-69	D	Necesita mejorar
0-59	F	Insuficiente



Criterios de Aprobación

- **Mínimo 70 puntos** en el proyecto para aprobar la semana
- El proyecto debe compilar sin errores de TypeScript
- Todas las rutas principales deben ser funcionales
- El dominio asignado debe estar claramente implementado



Formato de Entrega

Proyecto Semanal (Único Entregable)

- ☐ Repositorio con código fuente del proyecto
- ☐ BrowserRouter configurado correctamente
- ☐ Mínimo 5 rutas implementadas
- ☐ Al menos 2 rutas con parámetros dinámicos
- ☐ Layout principal con navegación
- ☐ Ruta protegida con redirección
- ☐ Página 404 personalizada
- ☐ Código TypeScript sin errores
- ☐ README con instrucciones y descripción del dominio



Adaptación por Dominio

El proyecto debe adaptarse al dominio asignado:

Dominio	Rutas Sugeridas
Biblioteca	/libros, /libros/:id, /autores, /prestamos, /mi-cuenta
Farmacia	/medicamentos, /medicamentos/:id, /ventas, /inventario
Gimnasio	/miembros, /miembros/:id, /rutinas, /clases, /horarios
Restaurante	/menu, /menu/:categoria, /reservas, /ordenes, /cocina
E-commerce	/productos, /productos/:id, /carrito, /checkout, /pedidos



Notas para el Instructor

- Verificar que las rutas protegidas redirijan correctamente
- Evaluar el uso de TypeScript en parámetros de ruta
- Comprobar que Outlet renderice los componentes hijos
- Revisar el manejo de rutas no encontradas (404)
- Verificar navegación programática con useNavigate

Semana 07 · React Router v6
